

Data types in c++ are:

Based data types: int, char, bool, float or double, void

Derived types (array, pointers reference stru-res, classes) object of the classes.

```
1 x = 6;
2 l1:
3 x += 2;
4 if (x < 10) goto l1;
5 if (x > 9) goto l2;
6 x = 11;
7 l2:
8 x = x + 4;
```

1. Логический тип данных.

это неподписанное значение, интерпретируемое как либо True, False.

2. Арифметические типы данных. : Int, float (double), char and bool.

3. Символьные типы данных.: Символьный тип (Char) — тип данных, предназначенный для хранения одного символа (управляющего или печатного) в определённой кодировке.

4. Размеры фундаментальных типов данных.

Минимальный размер для char — 8 бит, для short и int — 16 бит, для long — 32 бита, для long long — 64 бита.

5. Перечисления. (Enumerations in c++)

An **enum** is a special type that represents a group of constants (unchangeable values).

```
enum Level { LOW, MEDIUM, HIGH };
```

```
enum Level myVar;
```

```
enum Level myVar = MEDIUM;
```

5. Объявления и имена.

6. Инициализация в объявлении

Инициализация означает, что переменная запущена в работу, ей присвоено начальное значение, она инициализирована. (Initialization means that the variable is put into operation, it is assigned an initial value, it is initialized)

7. Понятие области видимости

Область видимости — зона доступности элементов в коде. (Visibility scope is the zone of accessibility of elements in the code)

8. Ключевое слово typedef.

Объя-ние typedef — это объявление с typedef в качестве класса хранения. (A typedef declaration is a declaration with typedef as the storage class.)

9. Тип данных void.

void — это тип, который допустимо указывать только как возвращаемое значение, которое указывает, что функция не возвращает значение, зато работу функции по-прежнему можно прерывать. (void is a type that can only be specified as a return value, which indicates that the function does not return a value, but the function can still be interrupted.)

10. Указатели, нулевой указатель.

Указатель — это переменная, в которой хранится адрес памяти объекта. (A pointer is a variable that stores the memory address of an object.)

нулевой указатель: это специальное значение, которое означает, что указатель ни на что не указывает (null)

11. Массивы. Инициализация массивов.

Массив — упорядоченный набор элементов, каждый из которых хранит одно значение, идентифицируемое с помощью одного или нескольких индексов. (An array is an ordered collection of elements, each of which stores a single value identified by one or more indices.)

12. Строковые литералы

Строковый литерал — это последовательность символов из исходного набора символов, заключенного в двойные кавычки (" "). (A string literal is a sequence of characters from the source character set enclosed in double quotation marks (" ") like: ("Hello world").

13. Константы.

В C++ константы — это переменные, значения которых не могут быть изменены после инициализации.

14. Ссылки

Ссылка в C++ -- это альтернативное имя объекта. Ссылку можно понимать как безопасный вариант указателя. (A reference in C++ is an alternative name for an object. A reference can be thought of as a safe variant of a pointer.) ex: (int number {5}; int &refNumber {number};)

15. Структуры.

Структура -- это производный тип данных, представляющий собой какую-либо конкретную определенную сущность, впрочем, как и класс. (A structure is a derived data type that represents a specific entity, just like a class.)

16. Выражения. Последовательность вычисления выражений.

Выражения — это последовательности операторов и операндов, используемые в следующих целях. Вычисление значения из операндов. Назначение объектов или функций. (Expressions are sequences of operators and operands used to: Compute a value from the operands Assign objects or functions.)

17. Приоритет операций.

::, [], (), ., ->	*, / %	>>, <<	==, !=, ?:
++, --,	+, -	<,<=, >, >=	=, *=, /=, %=, +=, -=

18. Арифметические операции.

19. Логические операции.

Логическое отрицание, НЕ ! R operator !(T a);

Логическое умножение, И && R operator &&(T a, S b);

Логическое сложение, ИЛИ || R operator ||(T a, S b);

20. Побитовые операции

a & b Побитовое И для a и b

a | b Побитовое ИЛИ для a и b

a ^ b Исключающее ИЛИ для a и b

~a Инверсия битов для a

a << b Побитовый сдвиг влево для a на b

a >> b Побитовый сдвиг вправо для a на b

21. Операции инкремента и декремента.

22. Операции работы с памятью new и delete.

Оператор new выделяет место в динамической памяти для объекта и возвращает указатель на этот объект. Оператор delete получает указатель на динамический объект и удаляет его из памяти. (The new operator allocates space in dynamic memory for an object and returns a pointer to that object. The delete operator receives a pointer to a dynamic object and deletes it from memory.)

23. Операции приведения типов. (Type casting operations.)

In c++ the data type casting is in four types: static_cast, dynamic_cast, const_cast, reinterpret_cast

Static_cast: static_cast <new_type> (exp); int n = 10; double nd = static_cast<double>(n);

dynamic_cast: dynamic_cast <new_type> (exp);

Animal* animalPtr = new Dog(); Dog* dogPtr = dynamic_cast<Dog*>(animalPtr);

const_cast: is used to modify the const or volatile qualifier of a variable.

const_cast <new_type> (exp);

const int n = 5; const int* ptr = &n; int* nonConstPtr = const_cast<int*>(ptr);

reinterpret_cast: The reinterpret_cast operator is used to convert the pointer to any other type of pointer

reinterpret_cast <new_type> (exp);

int n = 10; int* nptr = &n; char* charptr = reinterpret_cast<char*>(nptr);

24. Условные операторы if, if else, switch.

25. Операторы цикла for, while, do while.

26. Оператор goto.

Когда компилятор доходит до оператора goto он считывает идентификатор метки, и переходит к месту в коде, где расположена указанная метка. (When the compiler reaches a goto statement, it reads the label identifier and jumps to the location in the code where the specified label is located.)

27. Комментарии и отступы.

28. Понятие функции. Объявление функции.

Функция определяет действия, которые выполняет программа.

тип имя_функции(параметры){ инструкции}

29. Возврат значения из функции, оператор return.

Оператор return завершает выполнение функции и возвращает управление вызывающей функции

30.Перегрузка имен функций.

Перегрузка функций в языке программирования С++, определение функций с одним именем и разным набором параметров.

31.Правила разрешения перегрузки в случае одного аргумента.

If the collection of the parameters are the same so the rule to make the overloading function so we only change the datatype of the parameter of the function so there will no error.

32.Правила разрешения перегрузки в случае нескольких аргументов.

For many arguments, we also use the datatype of them.

33.Аргументы функции по умолчанию.

В программировании, аргумент по умолчанию является аргументом функции, который при вызове необязательно указывать. int my_func(int a, int b, int c=12);

34.Функции с переменным числом аргументов.

```
int sum(int n, ...){ int result = 0; va_list
factor;    ...}

printf("%d \n", sum(4, 1, 2, 3, 4));
```

35.Указатели на функции.

Указатель на функцию (function pointer) хранит адрес функции. тип (*имя_указателя) (типы_параметров);

36.Общая схема программы. Заголовочные файлы.

Заголовочные файл - содержимое которого автоматически добавляется препроцессором в исходный текст в том месте, где располагается некоторая директива.

```
int sum(int a, int b){    return a + b;} #include
"sum.h"
```

37.Компоновка. Правило одного определения.

Правило одного определения (One Definition Rule, ODR) —Назначение ODR состоит в том, чтобы в программе не могло появиться два или более конфликтующих между собой определения одной и той же сущности.

38.Классы и объекты. Переменные и функции члены классов.

Класс — это тип данных, созданный пользователем. Он содержит разные свойства и методы. Объект — это экземпляр класса

39.Управление режимом доступа к членам классов.

```
public (Открытый доступ).
protected (Защищённый доступ.
private (Закрытый доступ).
```

40.Конструкторы класса.

Конструкторы имеют имена, совпадающие с именами классов, и не имеют возвращаемых значений. Можно определить столько перегруженных конструкторов.

41.Статические члены класса.

Статические члены класса в ООП используются для хранения данных или методов, которые принадлежат всему классу, а не отдельным объектам.

```
class Counter { public: static int count; Counter() { count++; }
int Counter::count = 0;
```

```
std::cout << Counter::count<< endl;
```

Статические методы:

Описание: Статический метод (static) работает только со статическими полями класса. Он не имеет доступа к нестатическим членам объекта (так как вызывается без создания объекта). static int add(int a, int b) { return a + b; } std::cout << "Сумма 3 и 5: " << Math::add(3, 5) << endl;

42.Копирование объектов класса.

Копирование объектов класса — это процесс создания нового объекта, который является копией существующего. В ООП существует два типа копирования: поверхностное (shallow copy) и глубокое (deep copy). Они

позволяют управлять тем, как данные объекта дублируются при создании копии. 1.Поверхностное копирование (Shallow Copy):Описание: Копируются только значения полей. Если поле является указателем или ссылкой, то копируется только адрес, а не содержимое по этому адресу.

```
int* ptr; Example(int value) { ptr = new int(value); }
Example obj1(10); Example obj2 = obj1;
cout << "obj1: " << *(obj1.ptr); cout << "obj2: " << *(obj2.ptr);
```

2.Глубокое копирование (Deep Copy): Создаётся новая область памяти для каждого указателя или ссылки, и данные копируются в неё. int* ptr; Example(const Example& other) { ptr = new int(*(other.ptr)); Example obj1(10); Example obj2 = obj1; std::cout << "obj1: " << *(obj1.ptr);

43.Константные функции-члены.

Константные функции-члены в ООП— это методы класса, которые не изменяют состояние объекта. Они гарантируют, что члены объекта не будут модифицированы при вызове этих функций. Такие функции полезны для обеспечения безопасности и предсказуемости кода, особенно при работе с константными объектами.

```
Example: int getValue() const { return value; }
```

44.Структуры и классы.

Структура (struct): 1. Структуры используются для объединения нескольких связанных переменных (полей) в одном объекте. 2.Это простая форма данных, более характерная для процедурного программирования. 3. В С++ структуры также могут содержать методы, конструкторы, деструкторы и другие возможности ООП.

Класс (class):1.Классы: это основа ООП. Они представляют собой более сложные объекты, объединяющие данные и функции. 2. Классы поддерживают инкапсуляцию, наследование и полиморфизм.

45.Определение функций членов класса.

Определение функций-членов класса — это процесс описания методов класса, которые выполняют определённые действия с данными объекта. В С++ функции-члены можно определить как внутри самого класса (встроенные функции), так и за его пределами.

```
Inside the class: int area() const { return width * height; }
```

```
Inside the class but init!: int area() const; int Rectangle::area() const { return width * height; }
```

46.Функции-операции класса.

Функции-операции класса в С++ позволяют перегружать стандартные операторы (+, -, *, =, ==, <<, >> и др.) для работы с пользовательскими объектами. Это позволяет использовать операторы так, как если бы они работали с встроенными типами. 1. Перегрузка операторов: Перегрузка операторов — это способ определения, как стандартные операторы должны работать с объектами класса. Example: Complex operator+(const Complex& other) { return Complex(real + other.real, imag + other.imag); }

47.Перегрузка унарных операций класса.

Унарные операторы — это операторы, которые применяются только к одному операнду. В С++ их можно перегрузить для работы с пользовательскими классами. К таким операторам относятся: + (унарный плюс), - (унарный минус), ++ (инкремент), -- (декремент), ! (логическое отрицание), * (разыменование), & (взятие адреса), ~ (побитовое отрицание) 1.

Основные правила перегрузки

1.Унарный оператор перегружается как метод класса или дружественная функция. 2.Если оператор перегружается как метод класса, он не принимает аргументов, так как действует только на текущий объект (this). 3.Если оператор перегружается как дружественная функция, он принимает один аргумент — объект класса. int value; Number(int v) : value(v) {} Number operator-() { return Number(-value); } void display() const { cout << "Val: " << value; } main function: Number num(10); num.display(); Number negNum = -num; negNum.display();

48.Перегрузка бинарных операций класса.

Бинарные операторы — это операторы, которые принимают два операнда. Примеры бинарных операторов, которые можно перегрузить: +, -, *, /, %, ==, !=, <, >, <=, >=, &&, ||, &, |, ^, <<, >>, = и т.д.

Перегрузка бинарных операторов позволяет реализовать удобное взаимодействие между объектами пользовательских классов.

1. Основные правила перегрузки

Метод класса: Если оператор перегружается как метод класса, он принимает только один аргумент, потому что первый операнд — это сам объект (this).

2. Дружественная функция: Если оператор перегружается как дружественная функция, она принимает два аргумента.

3. Семантика: Перегруженный оператор должен сохранять привычную семантику. Например, + должен означать сложение, а == проверку равенства. Еха: class MyClass { public: MyClass operator+(const MyClass& other);};

49. Дружественные функции класса.

Дружественные функции в C++ позволяют предоставлять внешним функциям или классам доступ к(private) и (protected) членам класса. Это полезно, когда фун-ия должна взаимодействовать с закрытыми данными класса, но при этом не является его членом. **Ос-ые хара-ки дружественных фу-ий** 1. Опр-ие дружественной функции: Дружественная функция объявляется внутри класса с использованием ключевого слова friend, но реализуется вне класса. 2. Доступ: Дружественная функция имеет доступ ко всем членам класса (приватным и защищённым). 3. Не является членом класса: Дружественная функция не принадлежит классу. Она вызывается без объекта класса. 4. Гибкость: Дружественные функции полезны для операций, которые требуют доступ к закрытым данным двух или более объектов.

50. Операция индексирования operator[].

Оператор индексирования operator[] позволяет перегружать доступ к элементам объекта класса по индексу. Это особенно полезно для реализации пользовательских контейнеров (например, массивов, векторов и т. д.).

const Return Type& operator[](int index) const;

Основные характеристики:

- 1. Часто используется для контейнеров.
- 2. Можно перегрузить для чтения и записи.
- 3. Возвращаемое значение зависит от типа данных.
- 4. Перегрузка для константных объектов.

51. Операция функционального вызова operator().

Оператор функционального вызова operator() позволяет обращаться к объекту класса так, как если бы он был функцией. Это делает объекты класса функционально подобными функциям. Такой подход широко используется в библиотеках стандартных шаблонов C++.

int operator()(int x) { return value + x; } call like: class name(10)

52. Операции инкремента и декремента.

Операции инкремента (++) и декремента (--) используются для увеличения или уменьшения значения на единицу. Эти операции могут быть перегружены для пользовательских классов, чтобы работать с их объектами аналогично встроенным типам.

ClassName& operator++(); ClassName& operator--();

53. Наследование классов. Производные классы.

Наследование в C++ — это механизм, позволяющий создавать новый класс (производный), основанный на существующем классе (базовом). Производный класс наследует свойства и методы базового класса, что позволяет повторно использовать код и добавлять новые функциональности.

class DerivedClass : [access specifier] BaseClass { };

54. Конструирование и уничтожение объектов при наследовании классов.

When inheriting classes, the construction of an object always occurs in the order of the inheritance hierarchy, meaning the base class constructor is called first, followed by the derived class constructor; conversely, destruction happens in the reverse order, with the derived class destructor being called first, then the base class destructor.

Base class constructor called first:

Derived class constructor called next:

Destruction order is reversed:

- When an object is destroyed, the derived class destructor is called first, followed by the base class destructor.

55. Контроль доступа при наследовании классов.

1. Уровни доступа к членам класса

public: Члены доступны везде, где доступен объект класса.

protected: Члены до-пны в самом кл-се и во всех его производных классах.

private: Члены до-ны то внутри самого класса, в ко-ом они определены.

a. Public наследование

public члены базового класса остаются public в производном классе.

protected члены базового класса остаются protected в произво-ом классе.

private члены базового класса недоступны в производном классе.

b. Protected наследование

public члены базового класса становятся protected в производном классе.

protected члены базового класса остаются protected в произв-ном классе.

private члены базового класса недоступны в производном классе.

c. Private наследование

public и **protected** члены базового класса становятся private в производном классе.

private члены базового класса недоступны в производном классе.

class Base { public: int publicVar = 1;

protected: int protectedVar = 2;

private: int privateVar = 3; };

class PublicDerived : public Base { public:

void show() { cout << "PublicDer: " << publicVar << ", " << protectedVar } };

class ProtectedDerived : protected Base { public:

void show() { cout << "ProtectedDerived: " << publicVar << protectedVar; } };

class PrivateDerived : private Base { public:

void show() { cout << "PrivateDerived: " << publicVar << protectedVar; } };

int main() { PublicDerived obj1; cout << obj1.publicVar; // Доступно ProtectedDerived obj2; // cout << obj2.publicVar; // Ошибка: protected PrivateDerived obj3; // cout << obj3.publicVar; // Ошибка: private}

56. Полиморфизм. Виртуальные функции.

Полиморфизм (polymorphism) — это понятие из объектно-ориентированного программирования, которое позволяет разным сущностям выполнять одни и те же действия.

1. Виды полиморфизма

Компиляционный (статический) полиморфизм

- Осуществляется во время компиляции.
- Достигается с помощью перегрузки функций и операторов.

Исполнительный (динамический) полиморфизм

- Осуществляется во время выполнения программы.
- Реализуется с помощью виртуальных функций в C++.

Виртуальные функции Это функция, объявленная в базовом классе с использованием ключевого слова virtual.

Производные классы могут переопределять виртуальные функ.

При использовании указателя или ссылки на базовый класс вызов соответствующей функции определяется во время выполнения программы (динамическое связывание).

57. Абстрактные классы.

Абстрактные классы в ООР - это базовые классы, ко-ые мож. наследовать, но нельзя реализовывать. То есть на их основе нельзя создать объект.

58. Множественное наследование. (Multiple inheritance.)

это механизм в C++, который позволяет производному классу наследовать функциональность сразу от нескольких базовых классов. Этот подход предоставляет большую гибкость, но также может привести к сложности и проблемам, таким как конфликт имен и алмазная проблема.

lass Base1 { // ...}; class Base2 { // ...};

class Derived : public Base1, public Base2 { // ...};

59. Виртуальное наследование.

это метод C++, который гарантирует, что только одна копия переменных-членов базового класса наследуется внучатыми производными классами .

```
class Base { };
class Derived1 : virtual public Base { };
class Derived2 : virtual public Base { };
class Final : public Derived1, public Derived2 { };
```

60.Статическое и динамическое приведение типов.

Динамическое приведение типов, в отличие от статического, выполняется во время выполнения программы.

dynamic_cast<тип_в_который_преобразуем>(преобразуемый_объект)

```
Ebook cppbook{"About C++", 350, 6};
Book* book = &cppbook; // указывает на объект Ebook
Ebook* ebook{dynamic_cast<Ebook*>(book)};
ebook->print(); in 24 question there is also the info about it.
```

61.Операция typeid.

Операция typeid используется для получения информации о типе объекта или переменной во время выполнения программы. Она является частью механизма RTTI (Run-Time Type Information — информация о типе во время выполнения) и предоставляет доступ к объекту класса type_info, который описывает тип. **typeid(expression)**

```
cout << "Type of a: " << typeid(a).name());
```

62.Определение шаблона класса.

это механизм обобщённого программирования, который позволяет создавать классы с параметрами, представляющими типы данных. Шаблоны классов упрощают создание универсального кода, который может работать с различными типами данных без необходимости их дублирования.

```
template <typename T> class ClassName { // Определение класса с использованием параметра T };
EXAMPLE: template <typename T>
class Box { private:   T value; // Универсальное значение public:   Box(T v) : value(v) {} // Конструктор
    T getValue() const { return value; } // Метод для получения значения
    void setValue(T v) { value = v; } // Метод для установки значения };
```

63.Конкретизация шаблона.

Конкретизация шаблона (Template Specialization) в C++ Конкретизация шаблона позволяет изменить поведение шаблона для определённых типов данных. Это полезно, когда для одного или нескольких типов требуется уникальная реализация, отличная от общей. В C++ существует два типа конкретизации шаблонов:

Полная специализация — когда шаблон класса или функции заменяется конкретной реализацией для определённого типа.

```
template <typename T> class Printer { public:   void print(T value) {
    cout << "Generic printer: " << value << endl;   } };
// Полная специализация для типа string
template <>
```

```
class Printer<string> { public:   void print(string value)
{   cout << "String printer: " << value << endl;   } };
```

Частичная специализация — когда шаблон класса или фу-ии заменяется для группы типов, но не для всех возможных значений параметров.

```
template <typename T, typename U>class Pair {public:   T first;   U second;
    Pair(T f, U s) : first(f), second(s) {}
    void print() {   cout << "Pair: (" << first << ", " << second << ")" << endl;   } };
// Частичная специализация для случая, когда оба типа одинаковы
template <typename T> class Pair<T, T>{ public:   T first;   T second;
    Pair(T f, T s) : first(f), second(s) {}   void print() {
    cout << "Same type Pair: (" << first << ", " << second << ")" << endl;   } };
```

64.Параметры шаблонов.

- 1. Типовые параметры (Type Parameters)
Типовой параметр — это самый распространённый тип параметров шаблонов. Он позволяет шаблонам работать с любыми типами данных. Типовые параметры объявляются как typename или class.
template <typename T> // Типовой параметр T
T add(T a, T b) { return a + b; }

2. Параметры по умолчанию (Default Template Arguments)
template <typename T = int> // Параметр T по умолчанию - int
T multiply(T a, T b) { return a * b; }

3. Несколько параметров шаблонов (Multiple Template Parameters)
template <typename T, typename U> // Два параметра шаблона T и U

- void printPair(T first, U second) { cout << "First: " << first << ", Second: " << second << endl; }

4. Параметры шаблонов как значения (Non-type Template Parameters)
template <typename T, int N> // Параметр N — целое число
T multiplyByN(T value) { return value * N; }

5. Шаблоны с переменным числом аргументов (Variadic Templates)
template <typename... Args> // Параметры могут быть любого типа
void print(Args... args) { (cout << ... << args) << endl; // Разворачиваем пакеты аргументов }

6. Шаблоны классов с параметрами по умолчанию
template <typename T = double, int N = 10> // Тип и значение по умолчанию

65.Определение функциональных шаблонов.

Функциональный шаблон — это функция, определённая с использованием шаблонного параметра (обычно с ключевым словом template), который позволяет её использовать с разными типами данных. Типы данных, с которыми работает функция, указываются при её вызове.
template <typename T> // Типовой параметр T
T add(T a, T b) { return a + b; }
This question is answered in 67, all the types and the arguments are there.

66.Аргументы функциональных шаблонов.

Аргументы функциональных шаблонов — это параметры, которые передаются в шаблон функции.
Типы аргументов функциональных шаблонов:

- 1. Типовые аргументы (Type arguments)
template <typename T> // Типовой параметр T
T add(T a, T b) { return a + b; }

2. Не-типы (Non-type arguments)
template <typename T, int N> // Аргумент N — целое число
T multiplyByN(T value) { return value * N; }

3. Переменные шаблоны (Variadic template arguments)
template <typename... Args> // Переменное количество аргументов
void print(Args... args) { (cout << ... << args); // Сводка всех аргументов }

67.Перегрузка функциональных шаблонов.

Перегрузка функц-ных шаблонов — это механизм, который позволяет определять несколько версий функций с одинаковым именем, но с разными параметрами. Шаблонные функции могут быть перегружены как с обычными функциями, так и с другими шаблонными функциями.

Правила перегрузки функциональных шаблонов

- 1. Шаблонная функция может быть перегружена обычной функцией.
2. Мож. перегрузить одну шаблонную фу другой шаблонной функцией.
3. Компилятор выбирает наиболее подходящую перегруженную функцию на основе переданных аргументов.
4. Для разрешения конфликтов можно использовать явную специализацию.

```
template <typename T>
void print(T value) {   cout << "Template function: " << value << endl; }
// Обычная функция
void print(int value) {   cout << "Ordinary function: " << value << endl; }
```

68.Параметры шаблонов по умолчанию.

```
template <typename T= int>
void print(T value) {   cout << "Template function: " << value << endl; }
```

69.Специализация шаблонов.

Специализация шаблонов в C++ позволяет создавать особые версии шаблонов для конкретных типов или значений параметров. Это важно, когда общая реализация шаблона не подходит для определённого случая, и требуется особая логика.
Виды специализации шаблонов

1. Полная специализация

Полностью определяет реализацию шаблона для конкретного типа или значения.
Exa: template <typename T> class Printer { public:
void print(T value) { cout << "Generic printer: " << value << endl; } };
// Полная специализация для типа string

```
template <>
class Printer<string> { public:    void print(string value)
{   cout << "String printer: " << value << endl;   }};

2. Частичная специализация (только для классов)
Определяет шаблон для частично заданных параметров
(например, комбинации типов или других условий). Exam:
template <typename T, typename U>class Pair {public:    T first;    U second;
    Pair(T f, U s) : first(f), second(s) {}
    void print() {   cout << "Pair: (" << first << ", " << second << ")" << endl;   }};
// Частичная специализация для случая, когда оба типа одинаковы
template <typename T> class Pair<T, T> { public:    T first;    T second;
    Pair(T f, T s) : first(f), second(s) {}   void print() {
        cout << "Same type Pair: (" << first << ", " << second << ")" << endl;   }};
```

70.Объявление пространств имен.

Пространства имен (namespaces) в C++ используются для объединения логически связанных классов, функций, переменных и других элементов, чтобы избежать конфликтов имен в больших проектах. Они также помогают организовать код и улучшить его читаемость.

```
namespace MyNamespace { int x = 10; void display() { cout << "Value of x: " << x << endl; } }
int main() { MyNamespace::display(); return 0; }
```

71.Использование пространств имен.

```
namespace Math { int add(int a, int b) { return a + b; } }
int main() { std::cout << "Сумма: " << Math::add(5, 7) << std::endl; }
```

72.Генерация исключений. Ключевое слово throw.

1. Ключевое слово throw используется для генерации исключения.

2. После throw указывается объект исключения, который может быть любого типа

```
if (b == 0) { throw std::invalid_argument("Деление на ноль!"); }
std::cout << "Результат: " << a / b << std::endl;
```

73.Обработка исключений. Ключевые слова try и catch.

Синтаксис обработки исключений:

1.try: Используется для обозначения блока кода, где может произойти исключение.

2.catch: Используется для перехвата и обработки исключений. Следует сразу за блоком try.

3. Блок catch должен содержать тип исключения, который он может обработать.

```
try { ....} catch{.....}, or try{ .....} catch{...} catch{...}
```

74.Обработка нескольких исключений.

```
try {
    // Код, который может вызвать исключения
} catch (const Тип1& e1) {
    // Обработка исключения типа Тип1
} catch (const Тип2& e2) {
    // Обработка исключения типа Тип2
} catch (...) {
    // Обработка всех остальных исключений
}
```

- 1. Логический тип данных.
- 2. Арифметические типы данных
- 3. Символьные типы данных.
- 4. Размеры фундаментальных типов данных.
- 5. Объявления и имена.
- 6. Инициализация в объявлении
- 7. Понятие области видимости
- 8. Ключевое слово typedef.
- 9. Тип данных void.
- 10. Указатели, нулевой указатель.
- 11. Массивы. Инициализация массивов.

- 12. Строковые литералы
- 13. Константы
- 14. Ссылки
- 15. Структуры
- 16. Выражения Последовательность вычисления выражений.
- 17. Приоритет операций.
- 18. Арифметические операции.
- 19. Логические операции.
- 20. Побитовые операции
- 21. Операции инкремента и декремента.
- 22. Операции работы с памятью new и delete.
- 23. Операции приведения типов. (Type casting operations.)
- 24. Условные операторы if, if else, switch.
- 25. Операторы цикла for, while, do while.
- 26. Оператор goto.
- 27. Комментарии и отступы.
- 28. Понятие функции. Объявление функции.
- 29. Возврат значения из функции, оператор return.
- 30. Перегрузка имен функций.
- 31. Правила разрешения перегрузки в случае одного аргумента.
- 32. Правила разрешения перегрузки в случае нескольких аргументов.
- 33. Аргументы функции по умолчанию.
- 34. Функции с переменным числом аргументов.
- 35. Указатели на функции.
- 36. Общая схема программы. Заголовочные файлы.
- 37. Компоновка. Правило одного определения.
- 38. Классы и объекты. Переменные и функции члены классов.
- 39. Управление режимом доступа к членам классов.
- 40. Конструкторы класса.
- 41. Статические члены класса.
- 42. Копирование объектов класса.
- 43. Константные функции-члены.
- 44. Структуры и классы.
- 45. Определение функций членов класса.
- 46. Функции-операции класса.
- 47. Перегрузка унарных операций класса.
- 48. Перегрузка бинарных операций класса.
- 49. Дружественные функции класса.
- 50. Операция индексирования operator[].
- 51. Операция функционального вызова operator().
- 52. Операции инкремента и декремента.
- 53. Наследование классов. Производные классы.
- 54. Конструирование и уничтожение объектов при наследовании классов.
- 55. Контроль доступа при наследовании классов.
- 56. Полиморфизм. Виртуальные функции.
- 57. Абстрактные классы.
- 58. Множественное наследование.(Multiple inheritance.)
- 59. Виртуальное наследование.
- 60. Статическое и динамическое приведение типов.
- 61. Операция typeid.
- 62. Определение шаблона класса.
- 63. Конкретизация шаблона.
- 64. Параметры шаблонов.
- 65. Определение функциональных шаблонов.
- 66. Аргументы функциональных шаблонов.
- 67. Перегрузка функциональных шаблонов.
- 68. Параметры шаблонов по умолчанию.
- 69. Специализация шаблонов.
- 70. Объявление пространств имен.
- 71. Использование пространств имен.
- 72. Генерация исключений. Ключевое слово throw.
- 73. Обработка исключений. Ключевые слова try и catch.
- 74. Обработка нескольких исключений.